

Project: Build a Data Mart in SQL

Course code: DLBDSPBDM01

Use case: Airbnb Accommodation Booking Platform

Field	Detail
Course of Study	BSc. Computer Science.
Student's Name	Kashish Paresh Gada
Matriculation No.	102202657
Tutor	Anna Androvitsanea
Submission	Conception Phase (Phase 1)
Date	7th July 2026

1. Problem Definition and Objective

Airbnb is an online accommodation platform that connects host who wants to offer lodging with guests who want to book it for staying temporarily while the platform itself handles all the booking and financial settlements between both of them that is the host and the user or the guest. Operating such an Airbnb service generates a lot of interrelated data like user profiles, accommodation, listings, availability, bookings, payments, payouts, reviews and messages amongst others. To manage this information in a non-relational way quickly leads to data redundancy update anomalies and inconsistent records which will undermine billing accuracy, along with reduced trust between users and the reliability of the service.

The main goal of this portfolio is to design and implement A normalized relational database that models the Airbnb use case end to end this conception phase establishes the foundation for further building upon. It captures the requirements of the systems and identifies the roles involved and actions they perform and translates these into a formal entity relationship model. We also have a data dictionary that explains what each entity does.

2. Requirements Analysis

2.1 Roles (User Groups)

Every person on the platform holds a single User account. An account can act in one or both of two functional roles. Platform-level tasks (collecting payment, releasing the payout to the host) are performed automatically by the system rather than by an end user.

Role	Description and responsibilities
Guest	A user who searches for and books accommodation, pays for the stay, communicates with hosts, leaves reviews and saves listings to wishlists.
Host	A user who lists and manages accommodation, sets prices, availability, amenities, photos, house rules and a cancellation policy, receives bookings and payouts, and reviews guests.
User (shared)	The common account from which Host and Guest are specialised; holds identity, contact and login data and may link social-network profiles. A single user can be both a host and a guest.

2.2 Actions Performed by Each Role

Guest actions:

- Guests can register an account and manage their personal profile and add social links.
- Guests can search and filter listings by their location, price, room type and amenity requirements.
- They can view the listings availability calendar and house rules and check if they fit them or not.
- Guests can create a booking for a date range and number of guests that are going to stay on the Airbnb.
- Guests can pay for the booking using a stored payment method.
- Messages can be exchanged with the host about a booking.
- The guest can write a review and add a rating after the stay.
- If a guest likes a property and wants to save that for future they can save it in the wish lists.
-

Host actions:

- A user can register as a host and maintain the host profile.
- Hosts can create and edit listings they can also include address property and room types.
- Hosts can manage availability calendars, prices, amenities photos and house rules so the guest can view it.
- Hosts can also add a cancellation policy to each listing.
- Hosts can receive and track the bookings and check the payouts.
- Hosts can communicate and message the guests and review them after a stay.
- Hosts can estimate and expected income from a listing.

2.3 Required Data and Functions

Required data:

- Account and identity data for users, hosts and guests, plus linked social profiles and stored payment/payout methods.
- Listing data with location (country, city, address), property and room type, amenities, photos, house rules and a per-date availability calendar.
- Transactional data: bookings, payments, host payouts and the fees/commissions charged on each booking.
- Feedback data: reviews, the users they concern and per-category ratings.
- Communication data: direct messages between users.

Required functions:

- Account and profile management for both roles.
- Listing search, filtering and availability checking.
- The full booking lifecycle (request, confirm, cancel, complete).
- Payment capture from the guest and payout to the host after arrival, with commission/fee handling.
- Mutual guest-host reviews and rating aggregation.
- Messaging and display of a user's reputation through social contacts.

3. Entity Relationship Model

My ER diagram below (drawn using draw.io) presents the complete ER model as a relational schema. Each entity is shown as a table listing its attributes with the MySQL data type; primary keys are marked PK (underlined) and foreign keys FK. Relationships are drawn as connectors using crow's-foot cardinality, where a single bar denotes the “one” side and a fork denotes the “many” side. The model contains 25 entities, which together with the four junction tables that resolve the many-to-many and recursive relationships yields 29 tables at implementation.

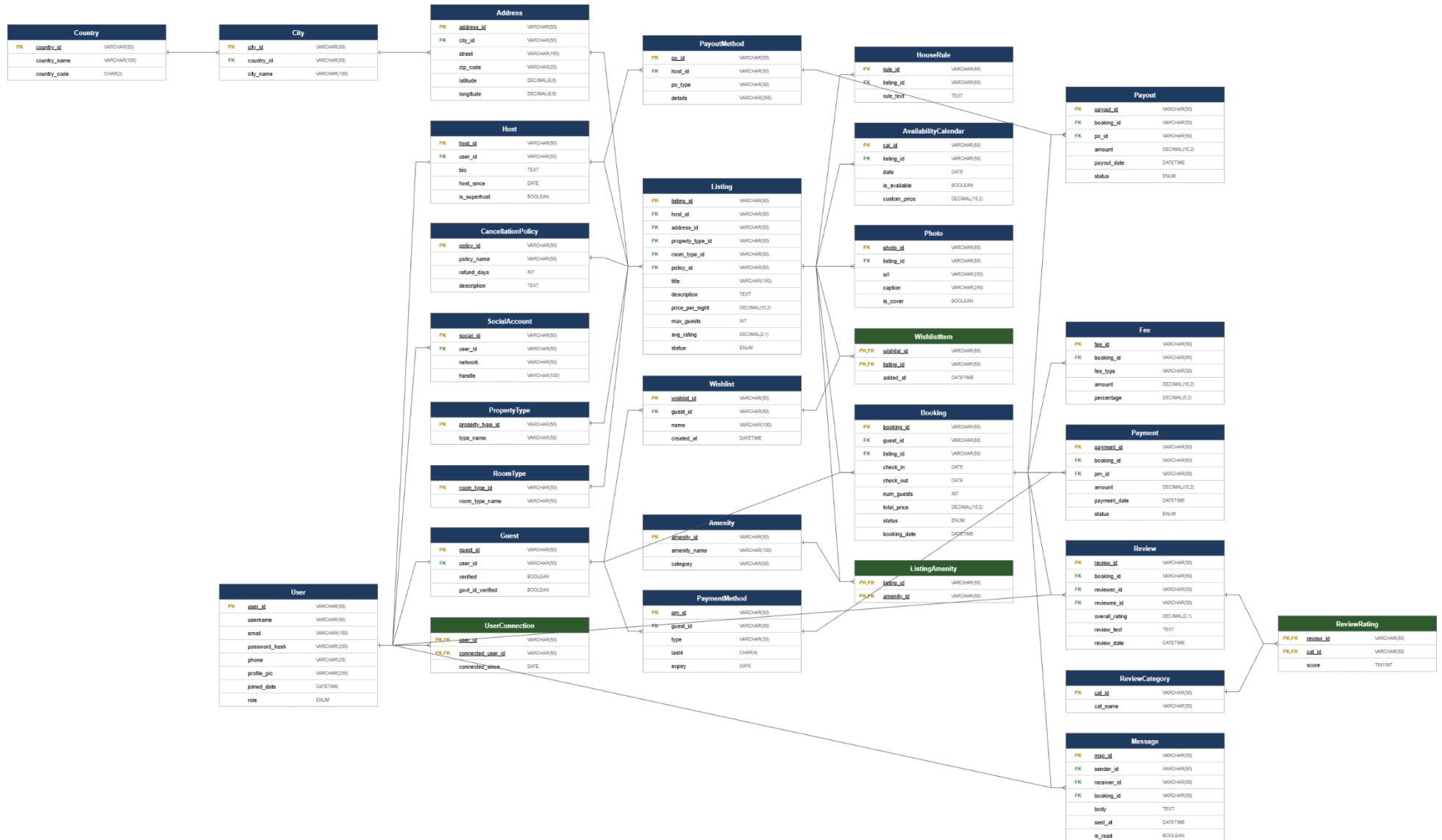


Figure 1: Entity Relationship Model (ERM)¹ – relational schema (crow's-foot notation)

¹ Self-drawn on draw.io

Triple (ternary) relationships – joins over three tables:

- books: a Guest books a Listing for dates held in the AvailabilityCalendar, realised through the Booking entity (Guest – Booking – Listing).
- reviews: a Review links a Booking to the reviewer and the reviewee, both of which are Users (User – Review – Booking).
- A third three-table join arises through Payment, which connects a Booking to the PaymentMethod used.

Recursive relationship:

- connected to: a User may be connected to other Users (social contacts), a many-to-many relationship of the User entity with itself, resolved by the UserConnection junction table.

Normalisation:

- The schema is normalised to 3NF. Look-up data (country, city, property type, room type, cancellation policy, amenity, review category) is factored into its own tables, and every many-to-many relationship is resolved with a junction table so that no repeating groups or transitive dependencies remain.

4. Data Dictionary

The data dictionary lists each entity with its attributes, MySQL data types and a short description. Primary keys are marked PK and foreign keys FK.

User

Base account shared by every person on the platform. Specialised into Host and Guest.

Attribute	Data Type	Key	Description
user_id	VARCHAR(50)	PK	Unique identifier for each user account.
username	VARCHAR(50)		Unique public display / login name.
email	VARCHAR(100)		Unique contact e-mail address.
password_hash	VARCHAR(255)		Securely hashed account password.
phone	VARCHAR(20)		Contact phone number.
profile_pic	VARCHAR(255)		Path/URL of profile picture.

Attribute	Data Type	Key	Description
joined_date	DATETIME		Date the account was created.
role	ENUM('GUEST','HOST','BOTH')		Indicates how the account is used.

Host

Role specialisation of User who lists accommodation.

Attribute	Data Type	Key	Description
host_id	VARCHAR(50)	PK	Unique host identifier.
user_id	VARCHAR(50)	FK	References the underlying User account.
bio	TEXT		Self-description shown on the host profile.
host_since	DATE		Date the user became a host.
is_superhost	BOOLEAN		Whether the host has Superhost status.

Guest

Role specialisation of User who books accommodation.

Attribute	Data Type	Key	Description
guest_id	VARCHAR(50)	PK	Unique guest identifier.
user_id	VARCHAR(50)	FK	References the underlying User account.
verified	BOOLEAN		Whether the guest identity is verified.
govt_id_verified	BOOLEAN		Whether a government ID was confirmed.

SocialAccount

External social-network profile linked to a user (Facebook etc.).

Attribute	Data Type	Key	Description
social_id	VARCHAR(50)	PK	Unique identifier of the linked profile.
user_id	VARCHAR(50)	FK	User who owns the linked profile.
network	VARCHAR(50)		Name of the social network.

Attribute	Data Type	Key	Description
handle	VARCHAR(100)		Profile handle or link.

PaymentMethod

A guest's stored means of payment.

Attribute	Data Type	Key	Description
pm_id	VARCHAR(50)	PK	Unique payment-method identifier.
guest_id	VARCHAR(50)	FK	Guest who owns the payment method.
type	VARCHAR(30)		e.g. credit card, PayPal.
last4	CHAR(4)		Last four digits (display only).
expiry	DATE		Expiry date of the instrument.

PayoutMethod

A host's account used to receive payouts.

Attribute	Data Type	Key	Description
po_id	VARCHAR(50)	PK	Unique payout-method identifier.
host_id	VARCHAR(50)	FK	Host who owns the payout method.
po_type	VARCHAR(30)		e.g. bank transfer, PayPal.
details	VARCHAR(255)		Masked account details / IBAN.

Country

Reference table of countries for normalised location data.

Attribute	Data Type	Key	Description
country_id	VARCHAR(50)	PK	Unique country identifier.
country_name	VARCHAR(100)		Full country name.
country_code	CHAR(2)		ISO 3166-1 alpha-2 code.

City

Reference table of cities, each belonging to one country.

Attribute	Data Type	Key	Description
city_id	VARCHAR(50)	PK	Unique city identifier.
country_id	VARCHAR(50)	FK	Country the city belongs to.
city_name	VARCHAR(100)		Name of the city.

Address

Physical address of a listing, linked to a city.

Attribute	Data Type	Key	Description
address_id	VARCHAR(50)	PK	Unique address identifier.
city_id	VARCHAR(50)	FK	City of the address.
street	VARCHAR(150)		Street and house number.
zip_code	VARCHAR(20)		Postal code.
latitude	DECIMAL(9,6)		Geographic latitude.
longitude	DECIMAL(9,6)		Geographic longitude.

Listing

An accommodation offered by a host (the central entity).

Attribute	Data Type	Key	Description
listing_id	VARCHAR(50)	PK	Unique listing identifier.
host_id	VARCHAR(50)	FK	Host who owns the listing.
address_id	VARCHAR(50)	FK	Address of the listing.
property_type_id	VARCHAR(50)	FK	Type of property.
room_type_id	VARCHAR(50)	FK	Type of room offered.
policy_id	VARCHAR(50)	FK	Cancellation policy that applies.
title	VARCHAR(150)		Headline shown to guests.
description	TEXT		Full textual description.

Attribute	Data Type	Key	Description
price_per_night	DECIMAL(10,2)		Base nightly price.
max_guests	INT		Maximum number of guests.
avg_rating	DECIMAL(2,1)		Average review rating.
status	ENUM('ACTIVE','INACTIVE')		Whether the listing is bookable.

PropertyType

Look-up of property types (apartment, house, loft, ...).

Attribute	Data Type	Key	Description
property_type_id	VARCHAR(50)	PK	Unique property-type identifier.
type_name	VARCHAR(50)		Name of the property type.

RoomType

Look-up of room types (entire place, private, shared).

Attribute	Data Type	Key	Description
room_type_id	VARCHAR(50)	PK	Unique room-type identifier.
room_type_name	VARCHAR(50)		Name of the room type.

CancellationPolicy

Cancellation/refund rules referenced by listings.

Attribute	Data Type	Key	Description
policy_id	VARCHAR(50)	PK	Unique policy identifier.
policy_name	VARCHAR(50)		Name (Flexible, Moderate, Strict).
refund_days	INT		Days before check-in for full refund.
description	TEXT		Human-readable policy text.

Amenity

Catalogue of amenities (Wi-Fi, pool, kitchen, ...).

Attribute	Data Type	Key	Description
amenity_id	VARCHAR(50)	PK	Unique amenity identifier.
amenity_name	VARCHAR(100)		Name of the amenity.
category	VARCHAR(50)		Grouping (e.g. Safety, Kitchen).

Photo

Photographs attached to a listing.

Attribute	Data Type	Key	Description
photo_id	VARCHAR(50)	PK	Unique photo identifier.
listing_id	VARCHAR(50)	FK	Listing the photo belongs to.
url	VARCHAR(255)		Path/URL of the image.
caption	VARCHAR(255)		Optional caption.
is_cover	BOOLEAN		Whether it is the cover photo.

HouseRule

House rules defined per listing.

Attribute	Data Type	Key	Description
rule_id	VARCHAR(50)	PK	Unique rule identifier.
listing_id	VARCHAR(50)	FK	Listing the rule applies to.
rule_text	TEXT		Description of the rule.

AvailabilityCalendar

Per-date availability and price overrides for a listing.

Attribute	Data Type	Key	Description
cal_id	VARCHAR(50)	PK	Unique calendar-entry identifier.
listing_id	VARCHAR(50)	FK	Listing the entry refers to.

Attribute	Data Type	Key	Description
date	DATE		Calendar date.
is_available	BOOLEAN		Whether the date is bookable.
custom_price	DECIMAL(10,2)		Optional price override.

Booking

A reservation of a listing by a guest (associative entity).

Attribute	Data Type	Key	Description
booking_id	VARCHAR(50)	PK	Unique booking identifier.
guest_id	VARCHAR(50)	FK	Guest who made the booking.
listing_id	VARCHAR(50)	FK	Listing that was booked.
check_in	DATE		Arrival date.
check_out	DATE		Departure date.
num_guests	INT		Number of guests.
total_price	DECIMAL(10,2)		Total amount for the stay.
status	ENUM('PENDING','CONFIRMED','CANCELLED','COMPLETED')		Booking lifecycle state.
booking_date	DATETIME		When the booking was made.

Payment

Payment made by a guest for a booking.

Attribute	Data Type	Key	Description
payment_id	VARCHAR(50)	PK	Unique payment identifier.
booking_id	VARCHAR(50)	FK	Booking being paid for.
pm_id	VARCHAR(50)	FK	Payment method used.
amount	DECIMAL(10,2)		Amount paid.
payment_date	DATETIME		Date of payment.
status	ENUM('PAID','REFUNDED','FAILED')		Payment state.

Payout

Disbursement to the host (released after arrival).

Attribute	Data Type	Key	Description
payout_id	VARCHAR(50)	PK	Unique payout identifier.
booking_id	VARCHAR(50)	FK	Booking being settled.
po_id	VARCHAR(50)	FK	Payout method used.
amount	DECIMAL(10,2)		Amount paid to host.
payout_date	DATETIME		Date the payout was released.
status	ENUM('PENDING','RELEASED')		Payout state.

Fee

Commission/fee lines attached to a booking.

Attribute	Data Type	Key	Description
fee_id	VARCHAR(50)	PK	Unique fee identifier.
booking_id	VARCHAR(50)	FK	Booking the fee relates to.
fee_type	VARCHAR(30)		Guest service / host service / cleaning.
amount	DECIMAL(10,2)		Fee amount.
percentage	DECIMAL(5,2)		Rate applied, where relevant.

Review

A review written about a stay (associative; links two users).

Attribute	Data Type	Key	Description
review_id	VARCHAR(50)	PK	Unique review identifier.
booking_id	VARCHAR(50)	FK	Booking being reviewed.
reviewer_id	VARCHAR(50)	FK	User who wrote the review.
reviewee_id	VARCHAR(50)	FK	User being reviewed.
overall_rating	DECIMAL(2,1)		Overall score (1.0-5.0).
review_text	TEXT		Free-text comment.
review_date	DATETIME		Date the review was posted.

ReviewCategory

Categories scored within a review (cleanliness, ...).

Attribute	Data Type	Key	Description
cat_id	VARCHAR(50)	PK	Unique category identifier.
cat_name	VARCHAR(50)		Name of the category.

Message

Direct message exchanged between two users.

Attribute	Data Type	Key	Description
msg_id	VARCHAR(50)	PK	Unique message identifier.
sender_id	VARCHAR(50)	FK	User who sent the message.
receiver_id	VARCHAR(50)	FK	User who received it.
booking_id	VARCHAR(50)	FK	Related booking (nullable).
body	TEXT		Message content.
sent_at	DATETIME		Timestamp of sending.
is_read	BOOLEAN		Whether it has been read.

Wishlist

A guest's saved collection of listings.

Attribute	Data Type	Key	Description
wishlist_id	VARCHAR(50)	PK	Unique wishlist identifier.
guest_id	VARCHAR(50)	FK	Guest who owns the wishlist.
name	VARCHAR(100)		Wishlist name.
created_at	DATETIME		Date created.

Associative / junction tables (created during implementation)

The following tables resolve the many-to-many and recursive relationships shown in the ERM.

ListingAmenity

Resolves the M:N relationship between Listing and Amenity.

Attribute	Data Type	Key	Description
listing_id	VARCHAR(50)	PK, FK	Listing offering the amenity.
amenity_id	VARCHAR(50)	PK, FK	Amenity offered.

ReviewRating

Resolves the M:N relationship between Review and ReviewCategory.

Attribute	Data Type	Key	Description
review_id	VARCHAR(50)	PK, FK	Review being scored.
cat_id	VARCHAR(50)	PK, FK	Category scored.
score	TINYINT		Score for that category (1-5).

WishlistItem

Resolves the M:N relationship between Wishlist and Listing.

Attribute	Data Type	Key	Description
wishlist_id	VARCHAR(50)	PK, FK	Wishlist containing the listing.
listing_id	VARCHAR(50)	PK, FK	Listing saved.
added_at	DATETIME		When it was added.

UserConnection

Resolves the recursive M:N 'connected to' relationship on User (social contacts).

Attribute	Data Type	Key	Description
user_id	VARCHAR(50)	PK, FK	First user.
connected_user_id	VARCHAR(50)	PK, FK	Connected user.
connected_since	DATE		Date the connection was made.